VHDL IMPLEMENTATION AND SIMULATION

1. Introduction

This section presents the design and simulation of a 32-bit CPU using VHDL. The system was developed by implementing individual components including the Arithmetic Logic Unit (ALU), Data Memory, Register File, Instruction Memory, and Control Unit. Each component was first tested independently using dedicated testbenches, after which they were integrated to form a complete CPU system.

All simulations were carried out using GHDL, and the resulting waveforms were visualized using GTKWave.

2. Design of Components

2.1 Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations based on a control signal. The operations implemented include:

- Addition
- Subtraction
- Bitwise AND
- Bitwise OR
- Set-less-than comparison

The ALU takes two 32-bit inputs and produces a 32-bit result along with a zero flag.

Simulation Results

The ALU was tested using the following operations:

- Add 2500 to 25000
- Subtract 37800 from 540250
- Perform AND on 53957 and 30000
- Perform OR on 746353 and 846465
- Compare 58847537 and 72464383

The waveform results confirm that the ALU correctly performs all required operations, with outputs matching expected values.

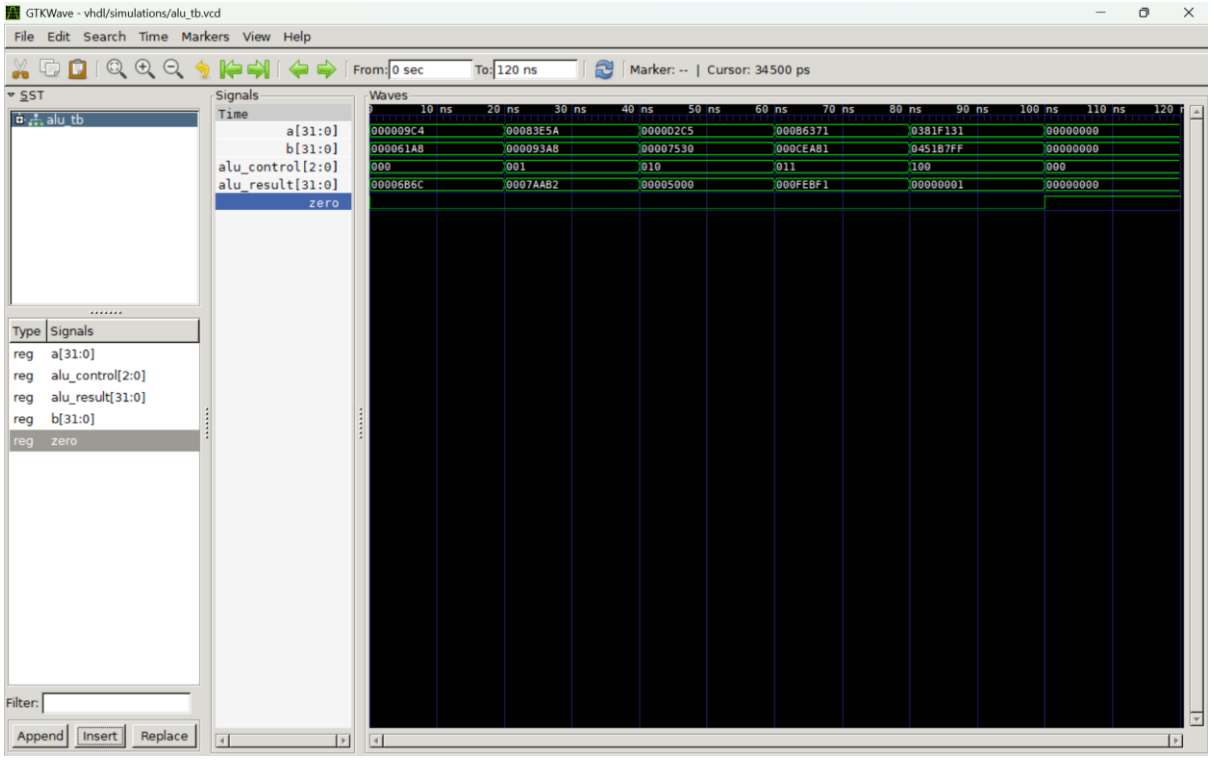

Figure 1: ALU waveform

2.2 Data Memory

The Data Memory module supports read and write operations controlled by `mem_read` and `mem_write` signals.

Test Cases Implemented

- Write 1024 into memory address 2 and read it back
- Write 429496 into memory address 4 and read it back

Simulation Results

The waveform shows that:

- Data is correctly written into specified memory locations
- The same data is successfully retrieved during read operations

This confirms proper memory functionality.

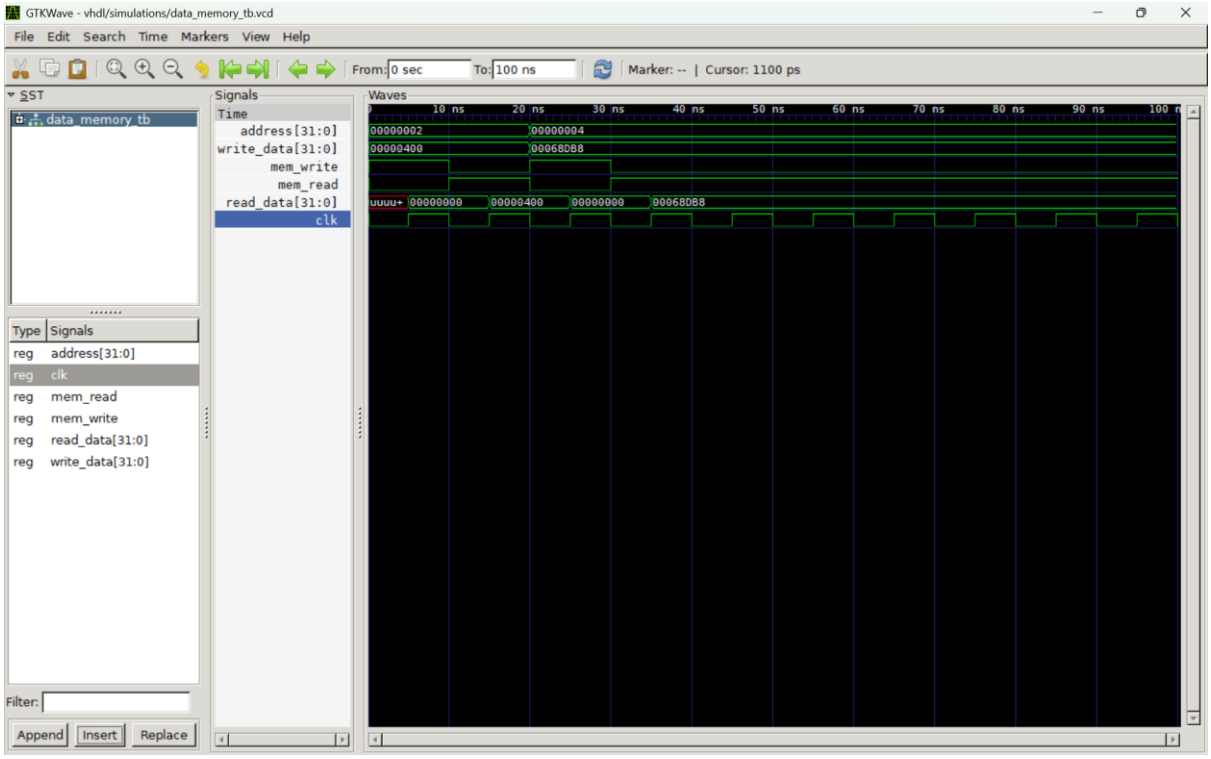

Figure 2: Data memory read/write operation

2.3 Register File

The Register File consists of multiple registers that support simultaneous read and write operations.

Test Cases Implemented

- Register 1: 1934858
- Register 3: 8558447
- Register 5: 203848544
- Register 7: 20670420

The values were written into the respective registers and later read back.

Simulation Results

The waveform confirms:

- Correct writing of data into registers
- Accurate retrieval of stored values

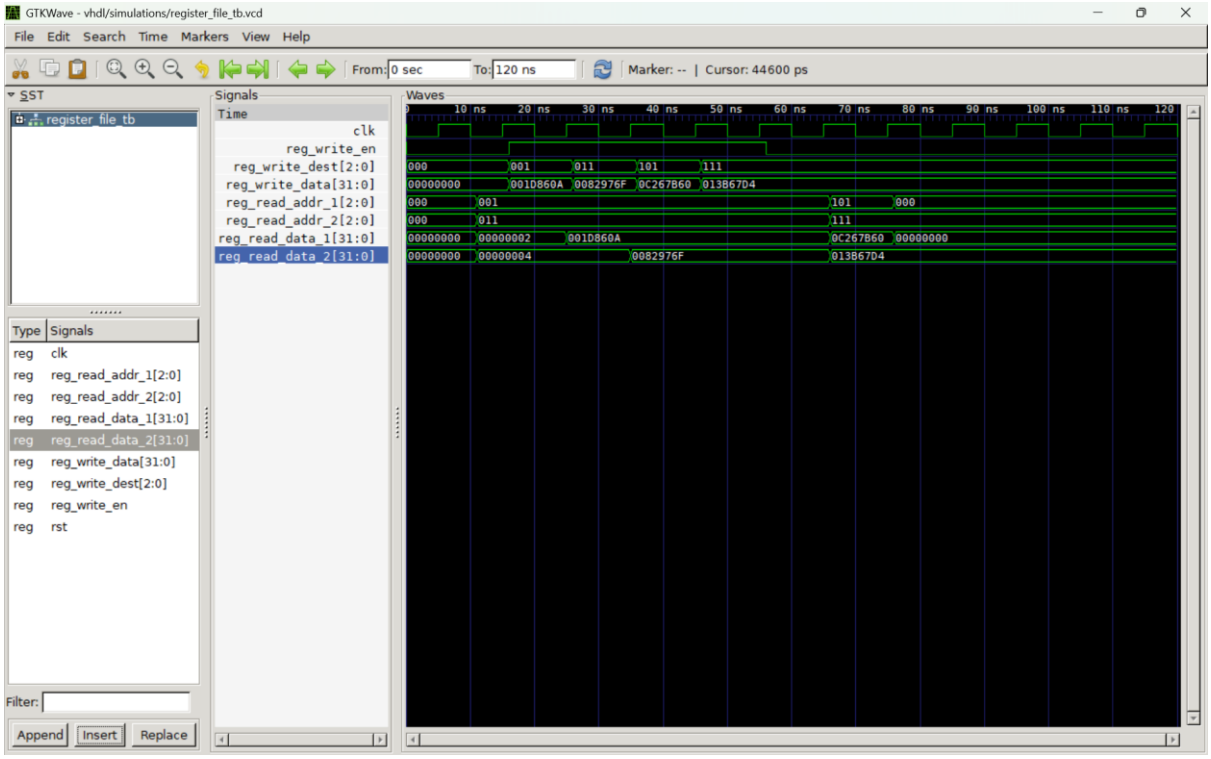

Figure 3: Register file simulation

2.4 Instruction Memory

The Instruction Memory stores 32-bit MIPS instructions and outputs them based on the input address.

Instructions Implemented

- Address 0: `add $t0, $t1, $t2`
- Address 2: `sub $t2, $t2, $t3`
- Address 4: `and $t1, $t2, $t0`
- Address 6: `or $t2, $t3, $t1`

Simulation Results

The waveform shows that:

- Each address correctly outputs its corresponding instruction
- Instruction sequencing is functioning properly

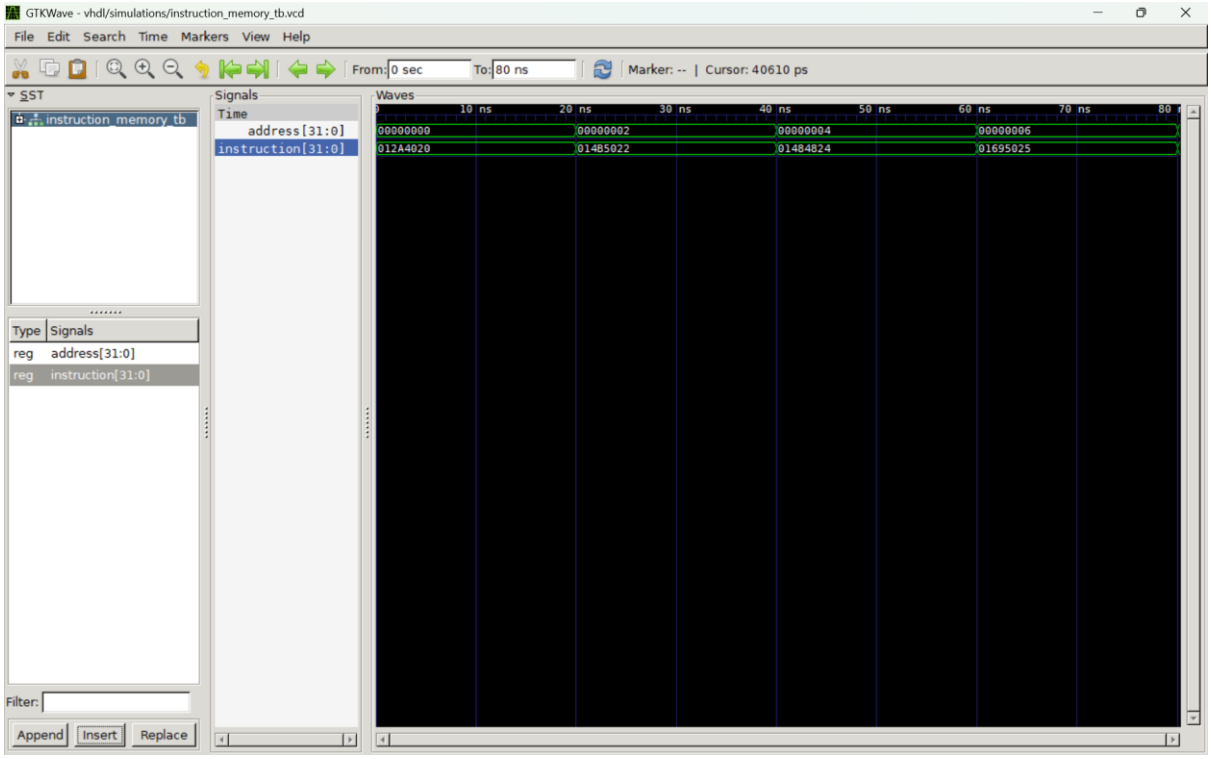

Figure 4: Instruction Memory Output

2.5 Control Unit

The Control Unit generates control signals based on the instruction opcode.

Instructions Tested

- add
- sltiu (sliu)
- j
- jal
- lw
- sw
- beq
- addi

Simulation Results

The waveform confirms that:

- Control signals such as `reg_write`, `mem_read`, `mem_write`, `branch`, and `jump` are correctly generated
- Different instructions produce the appropriate control signal combinations

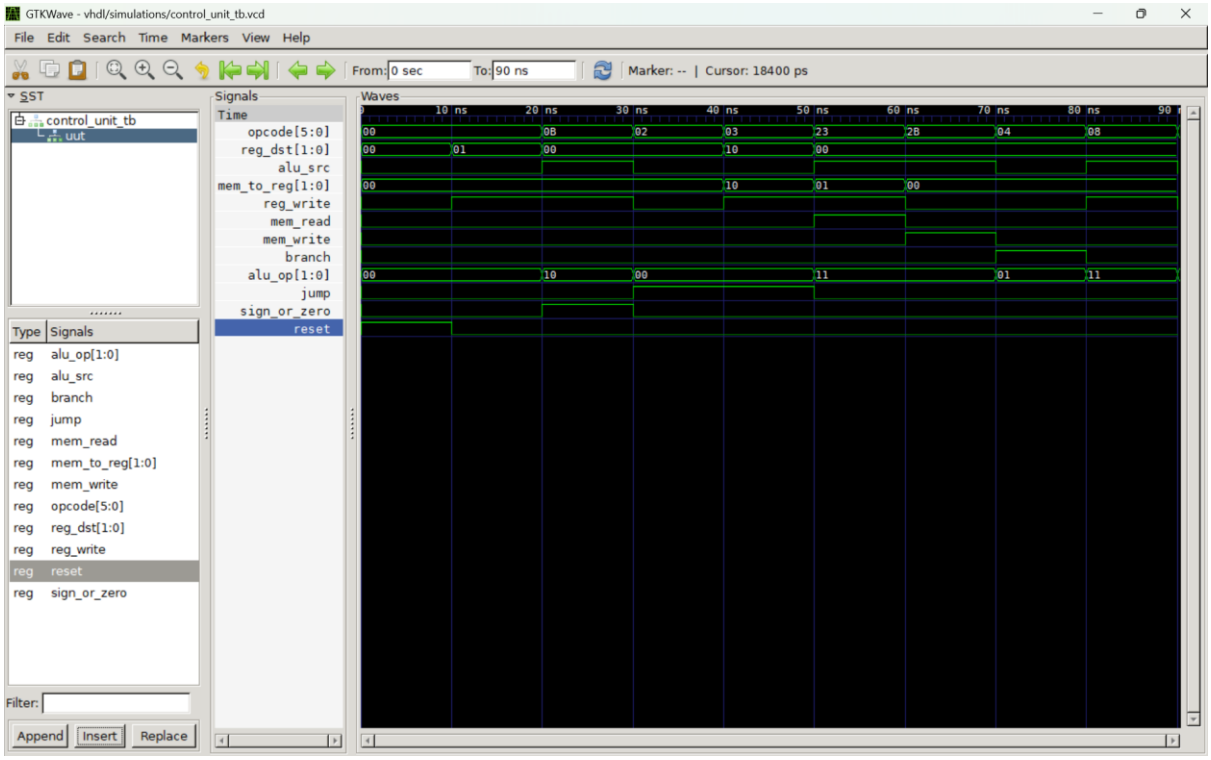

Figure 5: Control unit signals

3. CPU Integration

All individual components were integrated to form a complete 32-bit CPU.

The CPU connects:

- Instruction Memory → Fetch stage
- Register File → Operand supply
- ALU → Execution
- Data Memory → Memory access
- Control Unit → Control signal generation

4. CPU Simulation Results

The CPU was simulated using a testbench that executed multiple instructions sequentially.

The waveform shows:

- **Program Counter (PC)** incrementing correctly
- **Instruction flow** changing over time
- **ALU results** updating based on operations
- **Register values** being read and written
- **Clock signal** driving the system

These results confirm that the CPU executes instructions and that all components interact correctly.

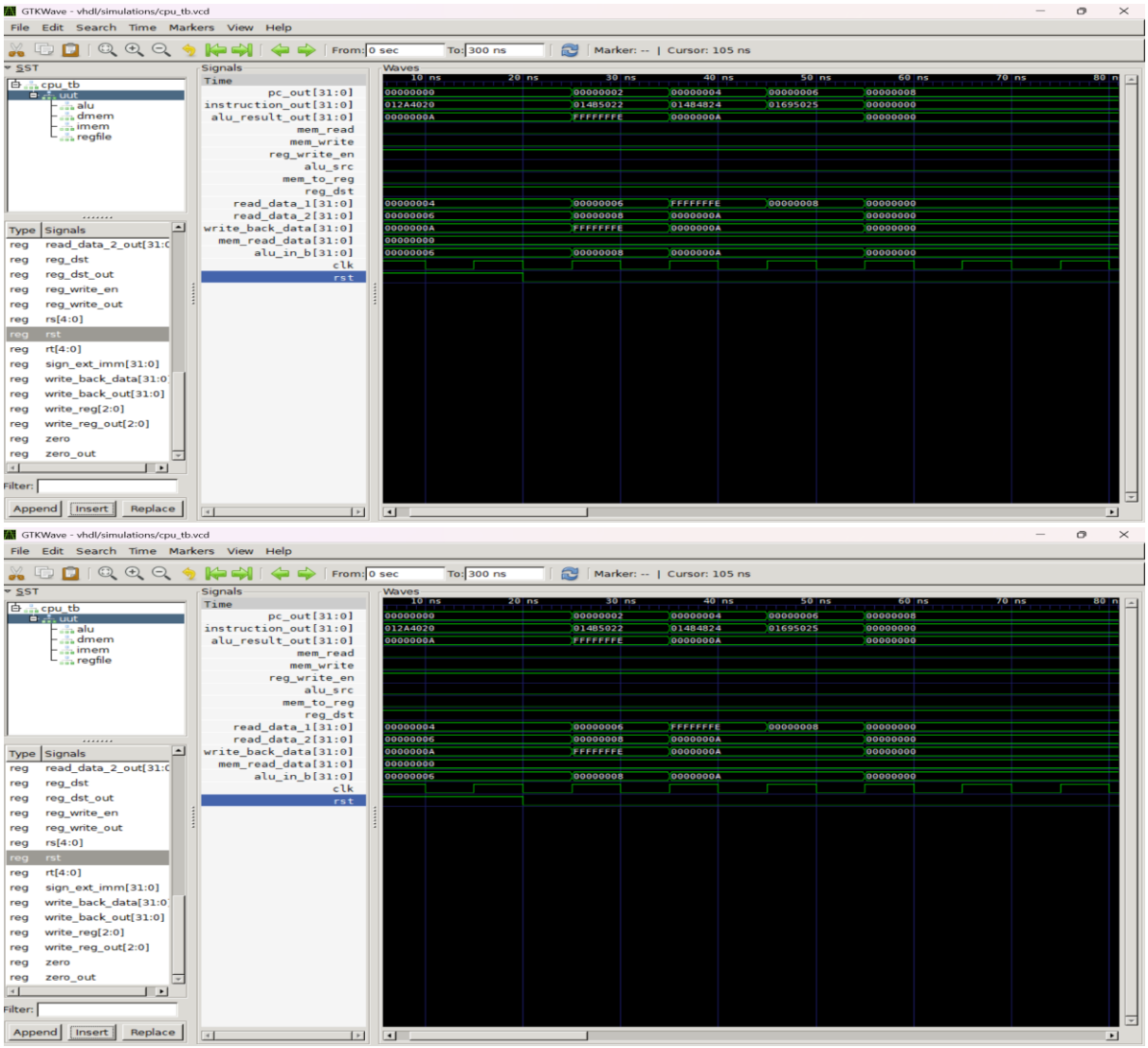

Figure 6: CPU execution waveform

5. Conclusion

The 32-bit CPU was successfully implemented using VHDL. Each component was verified individually through simulation, and the integrated CPU demonstrated correct instruction execution and data flow.

The system meets the required functional specifications and demonstrates correct modular integration of all CPU components

6. Contributors

Tetteh Emmanuel Etornam — ID: 22012489

Mark Kwamena Ekumah — ID: 22163947